

Implementation Roadmap

Project: Aura Promptly

Version: 1.1

Date: 2026-05-14

Status: Baselined (updated post-review)

Change Log

Version	Date	Changes
1.0	2026-05-14	Initial baseline
1.1	2026-05-14	Phase 1b (Basic CI/CD) added immediately after Phase 1; Phase 3 updated to Aurora pgvector (cost revised from +\$172 to +\$10–15/month); Phase 4 Agent demo redefined as Research Assistant with <code>search_knowledge_base</code> Lambda; Phase 7 GCP auth updated to direct STS federation; Phase 8 observability updated to two-layer; Phase 11 CI/CD scoped to full pipeline only; recreate runbook updated with KB sync poll-and-wait

Roadmap Philosophy

Each phase is:

- **Self-contained:** Deployable and demonstrable on its own
- **Incremental:** Builds on the previous phase without breaking it
- **Documented:** Every step written for a novice
- **Destroyable:** Can be torn down after the phase without losing progress (code stays in Git)

The phases are **not** meant to be completed in one sitting. The typical workflow is:

```
Start work session → terraform apply → work/demo → terraform destroy → commit
```

Phase Overview

Phase	Name	Prerequisites	Est. Hours	Monthly Cost Added
Phase 0	Requirements & Design	None	Complete 	\$0

Phase	Name	Prerequisites	Est. Hours	Monthly Cost Added
Phase 1	Hub Foundation	AWS account, GitHub repo, TF Cloud	4–6h	~\$45
Phase 1b	Basic CI/CD Pipeline	Phase 1, GitHub repo public	1–2h	\$0
Phase 2	Bedrock Inference + LiteLLM Routing	Phase 1	2–3h	~\$5 (Bedrock usage)
Phase 3	RAG Pipeline (Knowledge Base)	Phase 2	3–4h	+~\$10–15 (Aurora pgvector)
Phase 4	Bedrock Agent (Research Assistant)	Phase 3	3–4h	~\$2
Phase 5	AWS Spoke + Sample App	Phase 1	3–4h	~\$5
Phase 6	Azure Spoke + Sample App	Phase 1, Azure account	4–5h	~\$3
Phase 7	GCP Spoke + Sample App	Phase 1, GCP account	4–5h	~\$2
Phase 8	Observability (CloudWatch + OTel → Grafana)	Phase 1	3–4h	~\$3–5
Phase 9	OpenAI + Vertex AI Integration	Phase 2, API keys	2–3h	Usage-based
Phase 10	Guardrails Hardening	Phase 2	2–3h	~\$2
Phase 11	Full CI/CD (Infracost, tfsec, all workspaces)	All phases	4–6h	\$0
Phase 12	Cost Dashboard + Final Documentation	Phase 11	2h	\$0

Phase 0 — Requirements & High-Level Design

Status: Complete

Deliverables:

- ✓ [docs/01_requirements_fr_nfr.md](#)
- ✓ [docs/02_use_cases.md](#)
- ✓ [docs/03_high_level_design.md](#)
- ✓ [docs/04_adr_technology_decisions.md](#)
- ✓ [docs/05_cost_estimates.md](#)

- [docs/06_implementation_roadmap.md](#) (this document)
 - [docs/07_open_questions.md](#)
-

Phase 1 — Hub Foundation

Goal: Deploy the core AWS infrastructure for the hub: VPC, API Gateway, WAF, Cognito, ECS cluster, LiteLLM proxy (no AI calls yet), Aurora, Redis, Secrets Manager.

Deliverables:

- [infra/hub/](#) — Terraform module
- [infra/hub/README.md](#) — step-by-step deployment guide
- LiteLLM proxy running on pinned `-stable` image tag, health-check passing
- API Gateway returning 401 for unauthenticated requests
- `.gitignore` covering `*.tfvars`, `terraform.tfstate*`, `.terraform/`, `override.tf`
- `SECURITY.md` and `CODEOWNERS` files committed

Low-Level Design topics (to be written before implementation):

- VPC CIDR design and subnet layout
 - ECS task definition and IAM role
 - Aurora Serverless cluster configuration (shared: LiteLLM schema + KB schema)
 - Secrets Manager secret structure
 - API Gateway resource and method definitions
 - Cognito User Pool configuration (Azure API key issuance only)
-

Phase 1b — Basic CI/CD Pipeline

Goal: Establish a working GitHub Actions pipeline from day one. Catches Terraform errors and secret leaks on every PR.

Deliverables:

- `.github/workflows/terraform-pr-basic.yml` — `gitleaks scan + terraform fmt -check + terraform validate + terraform plan` (hub workspace)
- `.github/workflows/terraform-deploy-basic.yml` — `terraform apply` (hub) on merge to main
- AWS OIDC role configured (Terraform resource in hub module)
- GitHub repository settings: secret scanning enabled, push protection enabled
- `SETUP.md` documenting required GitHub secrets/variables for Phase 1b

Why now: Having a pipeline from Phase 1 prevents infrastructure drift, catches Terraform errors early, and ensures secret scanning is active before any credentials are ever near the repo.

Phase 2 — Bedrock Inference + LiteLLM Routing

Goal: Connect LiteLLM to Amazon Bedrock Claude Haiku 3. Make a successful inference call via the API Gateway.

Deliverables:

- LiteLLM config YAML with Bedrock model route (credentials via `os.environ/` references only)
- Terraform: Bedrock model access (IAM policy)
- Smoke test: `curl` the API Gateway and get a model response
- LiteLLM spend dashboard shows the test request

Prerequisite action (manual — do this early, can take up to 24 hours):

- Enable Claude Haiku 3 and Titan Text Embeddings v2 model access in AWS Console → Amazon Bedrock → Model Access

Phase 3 — RAG Pipeline (Knowledge Base)

Goal: Deploy Bedrock Knowledge Base backed by Aurora Serverless pgvector. Ingest sample documents. Verify RAG retrieval.

Deliverables:

- `infra/hub/modules/knowledge_base/` — Terraform module
 - Aurora pgvector schema (`bedrock_kb`) and HNSW index pre-created
 - Bedrock Knowledge Base registered against Aurora cluster
- S3 bucket for source documents
- Sample documents (3–5 web articles on AI governance / financial services AI — relevant to the demo scenario)
- Sync script / GitHub Actions workflow
- LiteLLM route for `bedrock-kb` (retrieve-and-generate)
- Test: question answering with source attribution

Cost note: Aurora pgvector adds ~\$10–15/month active, ~\$0 idle (scale-to-zero). No separate destroy step needed between sessions — the KB schema persists on the shared Aurora cluster at negligible cost.

Phase 4 — Bedrock Agent (Research Assistant)

Goal: Deploy a Bedrock Agent that demonstrates a multi-step agentic workflow: querying the Knowledge Base and synthesising a structured report.

Demo scenario: User asks *"What are the key AI governance controls in our policy documents? Draft a 3-bullet board summary."*

- Agent step 1: Calls `search_knowledge_base` Lambda action group → retrieves relevant KB chunks
- Agent step 2: Synthesises chunks into a structured report (Claude Haiku ReAct loop)
- Agent step 3: Returns formatted board summary

Deliverables:

- `infra/hub/modules/bedrock_agent/` — Terraform module
- Lambda function: `search_knowledge_base` — calls Bedrock KB `retrieve` API

```
def handler(event, context):
    kb_id = os.environ["KNOWLEDGE_BASE_ID"]
    query = event["inputText"]
    response = bedrock_agent_runtime.retrieve(
        knowledgeBaseId=kb_id,
        retrievalQuery={"text": query},
        retrievalConfiguration={"vectorSearchConfiguration":
{"numberOfResults": 5}}
    )
    return {"results": [r["content"]["text"] for r in
response["retrievalResults"]]}
```

- Bedrock Agent with Claude Haiku 3
- LiteLLM config route: `bedrock/agent/{AGENT_ID}/{ALIAS_ID}` (native support — no custom adapter)
- Test: submit the demo question and observe the full ReAct trace

Phase 5 — AWS Spoke + Sample App

Goal: Deploy a sample application in a separate AWS account that demonstrates IAM-authenticated calls to the hub Bedrock Agent.

Deliverables:

- `infra/aws-spoke/` — Terraform module (ECS Fargate or Lambda)
- Cross-account IAM role (spoke trust policy → hub API Gateway)
- Sample app (Python FastAPI + Streamlit) — Research Assistant demo
- App Docker image in ECR
- Working end-to-end test: spoke app → hub API GW (IAM Sig V4) → LiteLLM → Bedrock Agent

Phase 6 — Azure Spoke + Sample App

Goal: Deploy a sample application on Azure Container Apps that calls the hub RAG chatbot with an API key.

Deliverables:

- `infra/azure-spoke/` — Terraform module (ACA + Azure Key Vault)
- API key created in LiteLLM, stored in Azure Key Vault (never in code)
- Sample app (Python FastAPI + Streamlit) — RAG chatbot demo
- Working end-to-end test: Azure app → hub API GW (API key) → LiteLLM → Bedrock KB

Azure resources (minimal footprint):

- Resource Group
- Azure Container Apps Environment (consumption plan — \$0 when idle)
- Azure Key Vault (for API key storage)
- Azure Container Registry (for app image)

Phase 7 — GCP Spoke + Sample App

Goal: Deploy a sample application on GCP Cloud Run that authenticates via GCP Workload Identity OIDC → AWS STS federation.

Auth flow:

```
Cloud Run service account
→ GCP metadata server OIDC token
→ AWS STS AssumeRoleWithWebIdentity
→ Temporary IAM credentials (15-min TTL)
→ IAM Sig V4 call to Hub API Gateway
```

Deliverables:

- `infra/gcp-spoke/` — Terraform module (Cloud Run + GCP Secret Manager)
- AWS IAM OIDC provider for `accounts.google.com` (Terraform resource)
- IAM role with trust policy scoped to GCP service account `sub` claim
- Sample app (Python FastAPI + Streamlit) — document summarisation demo
- Working end-to-end test: GCP app → STS token exchange → hub API GW (IAM Sig V4) → LiteLLM → Bedrock Haiku

GCP resources:

- GCP Project (enable billing, APIs: Cloud Run, IAM, Secret Manager)
 - Cloud Run service (serverless — \$0 when idle)
 - GCP Secret Manager (for any local GCP-side secrets)
 - Service Account with Workload Identity binding
-

Phase 8 — Observability (Two-Layer: CloudWatch + Optional Grafana)

Goal: Establish full observability with CloudWatch as the primary layer and Grafana Cloud as the optional demo layer.

Deliverables:

- **Layer 1 (always-on):**
 - CloudWatch Log Groups with structured JSON parsing
 - CloudWatch Metrics via EMF (Embedded Metric Format) from LiteLLM
 - 4 CloudWatch Dashboards (Platform Overview, Spend per Team, Model Usage, Guardrail Events)
 - CloudWatch Alarms: error rate >5%, P95 latency >5s, spend >\$50/\$100/\$200
- **Layer 2 (optional, `enable_grafana_export = true`):**
 - OTel Collector ECS sidecar (alongside LiteLLM task)
 - Grafana Cloud account and API token (stored in Secrets Manager)
 - OTel Collector config: CloudWatch → Loki, Prometheus, Tempo
 - 4 Grafana dashboards (same content as CloudWatch dashboards)

- LiteLLM OTel callback configured
- Test: generate 10 requests, verify all appear in both CloudWatch and Grafana (if enabled) within 60 seconds

Article/video note: Show both dashboards. Explicitly demonstrate disabling `enable_grafana_export` for the "regulated environment" scenario — CloudWatch dashboards remain fully functional.

Phase 9 — OpenAI + Vertex AI Integration

Goal: Add OpenAI GPT-4o-mini and Vertex AI Gemini Flash as model routes in LiteLLM. Configure fallback chain.

Deliverables:

- OpenAI API key created and stored in Secrets Manager (referenced via `os.environ/` in LiteLLM config)
 - GCP project with Vertex AI API enabled, service account key in Secrets Manager
 - LiteLLM config updated with `openai-mini` and `vertex-flash` routes
 - Fallback chain: `bedrock-haiku` → `openai-mini` → error
 - Test: disable Bedrock, verify request falls back to OpenAI automatically
 - LiteLLM spend dashboard shows costs for all three providers
-

Phase 10 — Guardrails Hardening

Goal: Configure Bedrock Guardrails and LiteLLM content filter callbacks. Test blocking behaviour.

Deliverables:

- Bedrock Guardrails resource (Terraform)
 - Guardrail policy: hate speech, violence, misconduct — all BLOCK
 - Denied topics: financial advice, medical advice
 - LiteLLM keyword deny callback (secondary filter for non-Bedrock models)
 - Test suite: 5 test prompts that should be blocked, 5 that should be allowed
 - CloudWatch + Grafana dashboard showing guardrail violation events
 - Blocked request returns correct HTTP 400 with reason code
-

Phase 11 — Full CI/CD (GitHub Actions)

Goal: Extend the basic Phase 1b pipeline to cover all workspaces, add cost and security scanning, and automate destroy/recreate.

Deliverables:

- `.github/workflows/terraform-pr-full.yml` — full PR pipeline:
 - gitleaks secret scan
 - `terraform fmt -check + terraform validate`
 - `terraform plan` (all workspaces)
 - Infracost diff (posts cost delta to PR comment)

- tfsec / checkov security scan
- `.github/workflows/terraform-deploy-full.yml` — apply on merge to main (all workspaces)
- `.github/workflows/terraform-destroy.yml` — manual destroy via workflow_dispatch (spokes first, hub last)
- `.github/workflows/kb-sync.yml` — manual Knowledge Base sync trigger with poll-and-wait
- Azure and GCP OIDC configuration (Terraform resources)
- `SETUP.md` updated with all required GitHub secrets/variables

Phase 12 — Cost Dashboard + Final Documentation

Goal: Complete cost visibility tooling and finalise all documentation.

Deliverables:

- Infracost `infracost.yml` configuration committed
- Infracost CI/CD PR comment working end-to-end
- `docs/08_low_level_design_hub.md` — detailed LLD for hub components
- `docs/09_low_level_design_spokes.md` — detailed LLD for spoke applications
- `docs/10_runbooks.md` — operational runbooks (deploy, destroy, recreate with KB poll-and-wait, sync KB, rotate API keys)
- `docs/11_demo_script.md` — step-by-step demo script for YouTube video

Suggested Work Session Structure

Because the full PoC spans multiple phases and sessions, follow this pattern for each work session:

Session Start:

1. `git pull` (get latest code)
2. `terraform -chdir=infra/hub apply -auto-approve` (redeploy hub, ~15 min)
3. If using RAG: `enable_knowledge_base=true`, re-run apply, then run KB sync + poll

During Session:

- Work on the current phase
- Test as you go
- Commit code changes often (gitleaks will scan on every PR)

Session End:

1. `git add -A && git commit -m "Phase X: ..."`
2. `git push`
3. `terraform destroy` (all workspaces, ~10 min)
4. Verify AWS Console shows no running resources tagged `aura-promptly`

Recreate Runbook (with KB sync poll-and-wait)


```

# Step 1: Deploy hub
terraform -chdir=infra/hub apply -auto-approve

# Step 2: Upload KB documents
aws s3 sync ./data/kb-documents/ \
  s3://$(terraform -chdir=infra/hub output -raw kb_bucket_name)/

# Step 3: Trigger KB sync and wait for completion
JOB_ID=$(aws bedrock-agent start-ingestion-job \
  --knowledge-base-id $(terraform -chdir=infra/hub output -raw kb_id) \
  --data-source-id $(terraform -chdir=infra/hub output -raw
kb_data_source_id) \
  --query 'ingestionJob.ingestionJobId' --output text)

echo "Waiting for KB sync (job: $JOB_ID)..."
while true; do
  STATUS=$(aws bedrock-agent get-ingestion-job \
    --knowledge-base-id $(terraform -chdir=infra/hub output -raw kb_id) \
    --data-source-id $(terraform -chdir=infra/hub output -raw
kb_data_source_id) \
    --ingestion-job-id $JOB_ID \
    --query 'ingestionJob.status' --output text)
  echo "  Status: $STATUS"
  [ "$STATUS" = "COMPLETE" ] && echo "KB ready." && break
  [ "$STATUS" = "FAILED" ] && echo "KB sync FAILED." && exit 1
  sleep 15
done

# Step 4: Deploy spokes (KB is ready)
terraform -chdir=infra/aws-spoke apply -auto-approve
terraform -chdir=infra/azure-spoke apply -auto-approve
terraform -chdir=infra/gcp-spoke apply -auto-approve

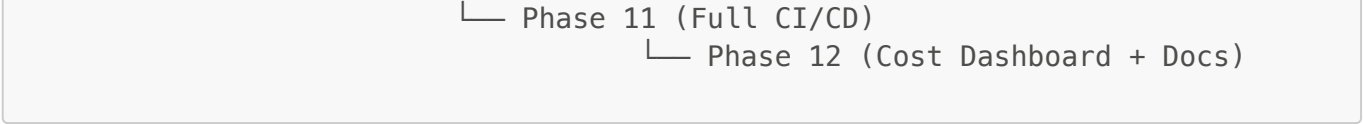
```

Dependency Graph

```

Phase 0 (Requirements/Design)
├── Phase 1 (Hub Foundation)
│   ├── Phase 1b (Basic CI/CD) ← start immediately after Phase 1
│   ├── Phase 2 (Bedrock Inference)
│   │   └── Phase 3 (RAG/KB – Aurora pgvector)
│   │       └── Phase 4 (Bedrock Agent – Research
Assistant)
│   │   └── Phase 9 (OpenAI + Vertex AI)
│   │       └── Phase 10 (Guardrails)
│   ├── Phase 5 (AWS Spoke)
│   ├── Phase 6 (Azure Spoke)
│   ├── Phase 7 (GCP Spoke – direct STS auth)
│   └── Phase 8 (Observability – two-layer)

```



- └─ Phase 11 (Full CI/CD)
- └─ Phase 12 (Cost Dashboard + Docs)

Phases 5, 6, 7, 8 can be worked in parallel after Phase 1.

Phases 9, 10 can be worked in parallel after Phase 2.

Phase 4 depends on Phase 3 (Agent uses the KB).